

FFF

(Forge's Flower Fight)

Unity 기반 화투 로그라이크 텍 빌딩 게임
프로젝트 결과 보고서

프로젝트명	FFF(Forge's Flower Fight)
장르	1 인용 로그라이크 텍 빌딩 카드 게임
개발 환경	Unity 6000.0.71f1 / C# / Universal Render Pipeline
팀	32204231 정혁진 32201142 김찬영 32211853 박지성
작성일	2026 년 5 월 30 일

목차

1. 프로젝트 개요
2. 개발 목표
3. 최종 산출물
4. 게임 콘셉트와 핵심 재미
5. 전체 게임 흐름
6. 주요 구현 기능
7. 기술 구조 및 아키텍처
8. 구현 규모 요약
9. 개발 결과 및 성과
10. 프로젝트의 강점
11. 한계 및 개선 방향
12. 향후 발전 방향
13. 종합 평가 및 결론

1. 프로젝트 개요

FFF(Forge's Flower Fight)는 한국 전통 카드인 화투와 섯다 족보 규칙을 기반으로 제작된 1 인용 로그라이크 덱 빌딩 게임이다. 플레이어는 매 전투마다 손패에서 2 장의 카드를 선택해 섯다 족보를 만들고, 조커와 장신구 효과를 조합해 적과 대결한다.

본 프로젝트는 단순한 카드 비교 게임이 아니라, 짧은 전투 단위의 전술 판단과 장기적인 덱 성장 구조를 결합한 전략형 게임을 목표로 한다. 플레이어는 전투 보상으로 카드, 조커, 장신구, 골드를 획득하며, 분기형 로그라이크 맵을 따라 최종 보스까지 도달해야 한다.

게임의 전투 규칙은 화투 카드 2 장을 제출해 섯다 족보를 판정하는 방식이며, 전투 외부에서는 맵 노드 선택, 상점 구매, 휴식, 보상 선택을 통해 런 전체의 방향성을 결정한다.

2. 개발 목표

1. 화투와 섯다 족보를 활용한 독자적인 카드 전투 시스템 구현
2. 타이틀, 메인, 맵, 전투, 상점, 휴식, 보상, 엔딩까지 이어지는 완결형 게임 루프 구현
3. 카드, 조커, 장신구를 통한 덱 빌딩 및 성장 구조 구현
4. 적 AI 와 적 기믹을 통해 전투별 차별성 제공
5. 조건과 효과를 조립하는 Modifier 기반 확장형 효과 시스템 구현
6. ScriptableObject 와 Resources 기반의 데이터 중심 설계 구축
7. UI, 사운드, 애니메이션을 포함한 플레이 가능한 빌드 산출

3. 최종 산출물

구분	내용
원본 프로젝트	Source Code/ 하위 Unity 프로젝트
실행 파일	실행 파일/My project.exe
주요 문서	README.md , FolderStructure (1).md , 개발 문서 URL
주요 씬	Boot, Title, Main, Stage, Battle, Shop, Rest, Treasure, Ending
핵심 시스템	전투 FSM, 덱 시스템, 섯다 족보 판정, Modifier, 맵 생성, 아이템, 적 AI, UI, 오디오

4. 게임 콘셉트와 핵심 재미

4.1 장르

FFF 는 로그라이크 덱 빌딩 카드 게임이다. 플레이어는 매 런마다 맵을 탐색하고, 전투와 보상을 반복하며, 자신의 덱과 아이템 구성을 강화한다.

4.2 핵심 재미

- 손패에서 어떤 2 장을 제출할지 결정하는 순간 전술
- 카드, 조커, 장신구를 선택하며 강한 조합을 만들어 가는 성장 전략
- 분기형 맵에서 위험과 보상을 비교하며 진행 경로를 선택하는 로그라이크 판단

4.3 한국적 소재 활용

일반적인 서양식 트럼프 카드가 아니라 월, 광, 띠, 열끗, 피 등 화투 카드의 구조를 사용하며, 광땡, 땡, 알리, 독사, 구땡, 장땡, 장사, 꽃, 망통 등 섯다 족보를 전투 공격력 계산의 중심 규칙으로 삼았다.

5. 전체 게임 흐름

```
BootScene -> TitleScene -> MainScene -> StageScene -> BattleScene -> ShopScene  
-> RestScene -> TreasureScene -> StageScene -> EndingScene
```

씬	역할
BootScene	전역 매니저와 씬 로더 초기화
TitleScene	게임 진입 화면
MainScene	새 게임 및 이어하기 선택

StageScene	로그라이크 맵 노드 선택
BattleScene	카드 선택, 리롤, 조커 사용, 전투 결과 처리
ShopScene	카드/장신구 구매 및 카드 제거
RestScene	휴식과 회복
TreasureScene	장신구 등 보상 획득
EndingScene	보스 클리어 후 엔딩 출력

6. 주요 구현 기능

6.1 전투 FSM 시스템

전투는 상태 머신 방식으로 구성되어 있으며, 중심 클래스는 `BattleManager` 이다. 전투 단계는 `BattleStart`, `TurnReady`, `TurnProceed`, `TurnEnd`, `BattleEnd` 로 나뉜다.

상태	역할
BattleStart	전투 데이터, 플레이어 전투 데이터, 적 데이터, 덱, Modifier 초기화
TurnReady	턴 시작, 카드 드로우, 리롤 횟수 초기화, 적 의도 결정
TurnProceed	플레이어 카드 선택, 조커 사용, 예상 공격력 갱신
TurnEnd	공격력 비교, 데미지 계산, 체력 반영, 버프 턴 감소
BattleEnd	승패 처리, 골드 지급, 보상 선택, 맵 복귀 또는 엔딩 전환

이 설계는 전투 단계가 명확하게 구분되어 디버깅이 쉽고, UI 가 상태 변경 이벤트를 구독해 필요한 시점에 갱신할 수 있다는 장점을 가진다.

6.2 덱 및 카드 시스템

덱 시스템은 `DeckSystem` 이 Facade 역할을 수행하며, 내부적으로 `CardPile`, `CardDrawHandler`, `CardSelectionHandler` 를 통합 관리한다. 기본 드로우 수는 5 장, 기본 리롤 횟수는 1 회, 기본 선택 카드 수는 2 장이다.

구성 요소	역할
-------	----

CardPile	드로우 더미, 손패, 선택 카드, 버린 카드 더미 관리
CardDrawHandler	카드 드로우, 리롤, 버린 더미 재활용 처리
CardSelectionHandler	선택 가능한 카드 수 제한 및 선택/해제 처리
DeckSystem	외부 시스템이 접근하는 통합 인터페이스

6.3 섯다 족보 판정 시스템

섯다 판정은 `SeotdaJudge` 와 `SeotdaHand` 를 중심으로 구현되어 있다. 선택한 화투 카드 2 장을 입력받아 족보를 판정하고, 해당 족보의 공격력 값을 반환한다.

- 광땡: 38 광땡, 18 광땡, 13 광땡
- 땡: 장땡부터 1 땡까지
- 특수 족보: 알리, 독사, 구땡, 장땡, 장사 등
- 곳: 9 곳부터 0 곳까지
- 망통

6.4 공격력 및 데미지 계산

선택 카드 2 장 -> `SeotdaJudge.Judge()` -> `SeotdaHand` 기본 공격력 산출 -> `ModifierManager` 를 통한 공격력 보정 -> 플레이어/적 공격력 비교 -> `DamageCal` 로 최종 피해량 산출 -> 체력 반영

같은 족보를 만들더라도 보유 아이템과 전투 조건에 따라 결과가 달라질 수 있도록 설계되어 있다.

6.5 Modifier 기반 효과 엔진

아이템과 적 기믹은 `BattleModifier` 로 표현되며, `Condition` 과 `Effect` 를 조립해 하나의 효과 단위로 구성한다.

구성 요소	역할
BattleModifier	조건, 효과, 지속 턴을 가진 전투 효과 단위
ModifierManager	등록된 Modifier 를 저장하고 계산 파이프라인에 적용
ModifierContext	현재 턴, 공격 주체, 카드, 플레이어, 적 등 조건 판정에 필요한 정보
IModifierCondition	효과 발동 조건 인터페이스
IModifierEffect	실제 수치 변경 방식 인터페이스

이 구조 덕분에 새 아이템이나 적 패턴을 추가할 때 전투 계산 로직을 직접 수정하지 않고 조건과 효과를 조합하는 방식으로 기능을 확장할 수 있다.

6.6 조커 시스템

조커는 전투 중 사용 가능한 액티브 아이템이다. `JokerItemBase` 를 상속한 개별 조커 클래스가 존재하며, 확인된 조커 구현은 `Joker_001` 부터 `Joker_008` 까지 총 8 종이다. 조커는 `ItemFactory` 를 통해 문자열 ID 기반으로 생성된다.

6.7 장신구 시스템

장신구는 전투에 지속 적용되는 패시브 아이템이다. `AccessoryItemBase` 를 상속한 개별 장신구 클래스가 존재하며, 확인된 구현은 `Accessory_001` 부터 `Accessory_020` 까지 총 20 종이다.

6.8 적 AI 및 적 기믹

적 시스템은 데이터, AI, 기믹이 분리되어 있다. 확인된 적 AI는 15종, 적 기믹도 15종이다. 적 AI는 전투 상황과 손패를 바탕으로 제출할 카드를 결정하고, 적 기믹은 Modifier를 통해 전투 규칙에 변화를 준다.

6.9 로그라이크 맵 시스템

맵 시스템은 `MapGenerator`, `MapData`, `MapNode`, `RoomType`으로 구성된다. 맵은 총 15층, 각 층 7열 구조이며 마지막에는 BossNode가 연결된다.

방 타입	가중치
Monster	45
Event	22
Elite	16
Rest	12
Shop	5

1층은 Monster, 9층은 Treasure, 15층은 Rest로 고정되며, 마지막 Rest층 이후 BossNode로 연결된다. 초반 층에서는 Elite와 Rest가 제한되고, 특정 제한 방 타입이 지나치게 인접하지 않도록 유효성 검사를 수행한다.

6.10 상점, 휴식, 보상 시스템

상점에서는 카드 구매, 장신구 구매, 카드 제거, 골드 소비 처리를 수행한다. 휴식은 전투 사이의 회복 공간으로 기능한다. 보상 시스템은 전투 보상, 이벤트 보상, TreasureScene 보상으로 나뉘며, 일반 전투는 50골드, 엘리트 전투는 100골드를 지급한다.

6.11 UI 및 오디오 시스템

UI 는 Title, Main, Map, Battle, Shop, Rest, Treasure, Ending, Common 영역으로 분리되어 있다. 오디오 시스템은 SoundManager, SoundCatalogSO, SoundIds, SoundBus 를 중심으로 BGM, SFX, UI 사운드를 분리 관리한다.

7. 기술 구조 및 아키텍처

7.1 전체 구조

```
Assets/Project/ ├── Art/ ├── Audio/ ├── Prefabs/ ├── Resources/ ├── Scenes/
                  ├── ScriptableObjects/ └── Scripts/
```

7.2 MVC/MVP 관점의 책임 분리

역할	폴더	설명
Model	Scripts/Data, Resources, ScriptableObjects	카드, 적, 아이템, 플레이어 데이터
View	Scripts/UI	화면 표시, 버튼, 카드 UI, HUD, 툴팁
Controller/Presenter	Scripts/Core, Scripts/Battle, Scripts/Map	씬 전환, 전투 진행, 맵 진행, 데이터 동기화

7.3 ScriptableObject 활용

카드 데이터, 아이템 데이터, 적 데이터, 적 AI, 적 기믹, 이벤트 채널, 사운드 카탈로그, 플레이어 마스터 데이터가 ScriptableObject 로 구성되어 있다. 이를 통해 코드와 데이터의 결합을 줄이고 Unity Inspector 에서 데이터를 조정할 수 있게 했다.

7.4 이벤트 채널 구조

전투와 UI 갱신은 ScriptableObject 이벤트 채널을 통해 느슨하게 연결된다. 카드 드로우 완료, 리롤 완료, 카드 선택 변경, 턴 상태 진입, 전투 종료 등의 이벤트가 발생하면 관련 UI 나 매니저가 이를 구독해 처리한다.

8. 구현 규모 요약

항목	규모
주요 플레이 씬	9 개
테스트 포함 씬 파일	11 개
조커 아이템 클래스	8 종
장신구 아이템 클래스	20 종
적 AI 클래스	15 종
적 기믹 클래스	15 종
맵 층 수	15 층
맵 열 수	7 열
기본 드로우 수	5 장
기본 리롤 수	1 회
기본 제출 카드 수	2 장

일반 전투 골드 보상	50 골드
엘리트 전투 골드 보상	100 골드

9. 개발 결과 및 성과

9.1 완성된 플레이 루프

프로젝트는 단일 전투 구현에 머무르지 않고, 타이틀부터 엔딩까지 이어지는 전체 게임 루프를 갖추었다. 플레이어는 새 게임을 시작하고, 맵에서 노드를 선택하며, 전투/상점/휴식/보상을 거쳐 성장하고 최종적으로 보스전과 엔딩에 도달할 수 있다.

9.2 전투 전략성 확보

전투는 손패 5 장 중 2 장을 선택하는 구조이므로 매 턴 선택지가 발생한다. 여기에 리롤, 조커 사용, 적 의도, 장신구 효과, 적 기믹이 결합되어 단순한 카드 숫자 비교보다 풍부한 판단 요소를 제공한다.

9.3 확장 가능한 효과 시스템 구현

Modifier 시스템을 통해 아이템과 적 기믹을 조건/효과 조합으로 만들 수 있게 되었다. 이는 향후 콘텐츠 추가에 특히 유리하다.

9.4 로그라이크 진행 구조 구현

시드 기반 맵 생성, 층별 고정 규칙, 방 타입 가중치, 방문/도달 가능 노드 저장, 보스 노드 연결 등 로그라이크 맵의 핵심 요소가 구현되었다.

10. 프로젝트의 강점

- **소재의 차별성:** 화투와 섯다 족보를 로그라이크 덱 빌딩에 결합했다.
- **명확한 상태 기반 전투 구조:** FSM 기반 전투 흐름으로 기능 확장과 디버깅이 쉽다.
- **아이템 및 적 기믹 확장성:** Modifier 시스템으로 조커, 장신구, 적 기믹을 통합 처리한다.
- **런 진행 데이터 관리:** PlayerDataSO 와 PlayerDataBattle 을 분리해 전투 중 임시 상태와 영구 데이터를 구분한다.
- **실제 플레이 가능한 산출:** 원본 코드뿐 아니라 실행 파일이 포함되어 결과물을 직접 확인할 수 있다.

11. 한계 및 개선 방향

11.1 프로젝트 이름과 빌드 메타데이터 정리

ProjectSettings 에서 productName 이 My project, companyName 이 DefaultCompany 로 남아 있다. 최종 제출 또는 배포를 고려하면 FFF 또는 Forge's Flower Fight 로 정리하는 것이 좋다.

11.2 ItemFactory 확장성

현재 ItemFactory 는 switch 문으로 모든 조커와 장신구를 직접 매핑한다. 아이템 수가 늘어날 경우 Dictionary 등록, ScriptableObject 기반 생성 전략, 리플렉션 기반 자동 탐색 등을 고려할 수 있다.

11.3 테스트 자동화 부족

셋다 족보 판정, 공격력/데미지 계산, Modifier 조건/효과 조합, 맵 생성 규칙, 아이템 ID 와 Factory 매핑 등은 자동화 테스트 가치가 높다.

11.4 데이터 검증 도구 필요

ID 기반 연결이 많기 때문에 카드 ID, 아이템 ID, 적 ID, SO 파일명, Factory 매핑, Resources 경로가 어긋나면 런타임 오류가 발생할 수 있다. Unity Editor 확장 도구를 통한 검증 기능을 추가하면 안정성이 크게 올라간다.

11.5 밸런싱 데이터 외부화

족보 공격력, 전투 보상 골드, 맵 방 타입 가중치 등을 ScriptableObject 또는 CSV 기반 데이터로 외부화하면 밸런싱 작업이 쉬워진다.

12. 향후 발전 방향

- 신규 조커, 장신구, 적, 보스 추가
- 이벤트 노드 스토리 확장
- 카드 변형 또는 강화 시스템 추가
- 난이도 단계 또는 챌린지 모드 도입
- 족보 완성 컷인, 조커 사용 연출, 전투 결과 연출 강화
- 카드/아이템/적 데이터 검증용 Unity Editor 도구 제작
- EditMode 및 PlayMode 테스트 자동화

13. 종합 평가 및 결론

FFF 프로젝트는 Unity 기반 캡스톤 프로젝트로서 완결형 게임 루프, 전투 시스템, 로그라이크 맵, 덱 빌딩, 아이템, 적 AI, UI, 사운드까지 폭넓은 기능을 구현했다. 특히 화투와 섯다라는 한국적 소재를 로그라이크 덱 빌딩 구조와 결합한 점이 인상적이며, 단순한 규칙 재현을 넘어 아이템과 Modifier 시스템을 통해 게임적 확장성을 확보했다.

전투 FSM, DeckSystem, ModifierManager, GameManager, MapGenerator 등 주요 시스템이 역할별로 분리되어 있어 구조적 완성도도 양호하다. 조커 8 종, 장신구 20 종, 적 AI/기믹 15 종 등 콘텐츠 규모도 캡스톤 프로젝트 결과물로서 충분히 풍부하다.

기술적으로는 Unity 6000.0.71f1, C#, ScriptableObject, Resources, FSM, 이벤트 채널, Modifier 패턴을 활용하여 기능별 책임을 나누고 확장 가능한 구조를 구축했다. 결과적으로 FFF는 기획, 구현, 데이터 구성, UI, 오디오, 빌드 산출까지 포함한 완성형 캡스톤 프로젝트로 평가할 수 있다.